

Chapter 4

Descriptive Analysis of Network Graph Characteristics

4.1 Introduction

In the study of a given complex system, questions of interest can often be re-phrased in a useful manner as questions regarding some aspect of the structure or characteristics of a corresponding network graph. For example, various types of basic social dynamics can be represented by triplets of vertices with a particular pattern of ties among them (i.e., triads); questions involving the movement of information or commodities usually can be posed in terms of paths on the network graph and flows along those paths; certain notions of the ‘importance’ of individual system elements may be captured by measures of how ‘central’ the corresponding vertex is in the network; and the search for ‘communities’ and analogous types of unspecified ‘groups’ within a system frequently may be addressed as a graph partitioning problem.

The structural analysis of network graphs has traditionally been treated primarily as a descriptive task, as opposed to an inferential task, and the tools commonly used for such purposes derive largely from areas outside of ‘mainstream’ statistics. For example, an overwhelming proportion of these tools are naturally graph-theoretic in nature, and thus have their origins in mathematics and computer science. Similarly, the field of social network analysis has been another key source, contributing tools usually aimed—at least originally—at capturing basic aspects of social structure and dynamics. More recently, the field of physics has also been an important contributor, with the proposed tools often motivated by analogues in statistical mechanics.

We present in this chapter a brief overview of some of the many such tools available, starting with summaries of vertex and edge characteristics, in Sect. 4.2, continuing with measures of network cohesion, in Sect. 4.3, and with methods for graph partitioning (aka ‘community detection’), in Sect. 4.4, and finishing with the topics of assortativity and mixing, in Sect. 4.5.

4.2 Vertex and Edge Characteristics

As the fundamental elements of network graphs are their vertices and edges, there are a number of network characterizations centered upon these. We discuss several such characterizations here in this section. Our presentation is broken down according to (i) those characterizations based upon vertex degrees, and (ii) those seeking to capture some more general notion of the ‘importance’ of a vertex—typically referred to as vertex centrality measures. We also discuss the extension of such concepts from vertices to edges.

4.2.1 Vertex Degree

Recall that the degree d_v of a vertex v , in a network graph $G = (V, E)$, counts the number of edges in E incident upon v . Given a network graph G , define f_d to be the fraction of vertices $v \in V$ with degree $d_v = d$. The collection $\{f_d\}_{d \geq 0}$ is called the *degree distribution* of G , and is simply a rescaling of the set of degree frequencies, formed from the original degree sequence.

The degree distribution for the karate club network is shown in Fig. 4.1, using a histogram.

```
#4.1 1 > library(sand)
      2 > data(karate)
      3 > hist(degree(karate), col="lightblue", xlim=c(0, 50),
      4 +   xlab="Vertex Degree", ylab="Frequency", main="")
```

We can see that there are three distinct groups of vertices, as measured by degree. The two most highly connected vertices correspond to actors 1 and 34 in the network, representing the instructor and administrator about whom the club eventually split. The next set of vertices consists of actors 2, 3, and also 33. Examination of our visualization of this network in Fig. 3.5 shows these actors to be among the closest to actors 1 and 34, respectively.

For weighted networks, a useful generalization of degree is the notion of vertex *strength*, which is obtained simply by summing up the weights of edges incident to a given vertex. The distribution of strength—sometimes called the weighted degree distribution—is defined in analogy to the ordinary degree distribution. To illustrate, vertex strength for the karate club network is also shown in Fig. 4.1.

```
#4.2 1 > hist(graph.strength(karate), col="pink",
      2 +   xlab="Vertex Strength", ylab="Frequency", main="")
```

For this network, the range of vertex strength stretches well beyond that of vertex degree, and the previously observed distinction among the three groups of vertices is lost. Nevertheless, both histograms arguably communicate important information about the network.

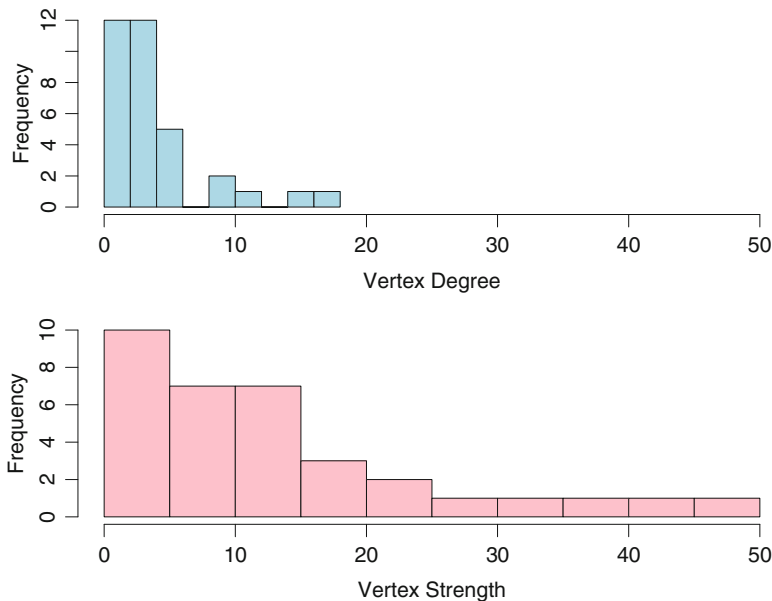


Fig. 4.1 Distributions of vertex degree (*top*) and strength (*bottom*) for the karate club network

Degree distributions can exhibit a variety of shapes. For a network of interactions among protein pairs in yeast, for example,

```
#4.3 1 > library(igraphdata)
      2 > data(yeast)
```

the shape is quite different from that of the karate club. In this case, the distribution of degrees associated with the

```
#4.4 1 > ecount(yeast)
      2 [1] 11855
```

edges among

```
#4.5 1 > vcount(yeast)
      2 [1] 2617
```

vertices is quite heterogeneous, as can be seen from examination of the histogram in Fig. 4.2.

```
#4.6 1 > d.yeast <- degree(yeast)
      2 > hist(d.yeast,col="blue",
      3 +       xlab="Degree", ylab="Frequency",
      4 +       main="Degree Distribution")
```

In particular, while there is a substantial fraction of vertices of quite low degree, of an order of magnitude similar to those of the karate network, there are also a non-trivial number of vertices with degrees at successively higher orders of magnitude.

Given the nature of the decay in this distribution, a log–log scale is more effective in summarizing the degree information.

```
#4.7 1 > dd.yeast <- degree.distribution(yeast)
2 > d <- 1:max(d.yeast)-1
3 > ind <- (dd.yeast != 0)
4 > plot(d[ind], dd.yeast[ind], log="xy", col="blue",
5 +     xlab=c("Log-Degree"), ylab=c("Log-Intensity"),
6 +     main="Log-Log Degree Distribution")
```

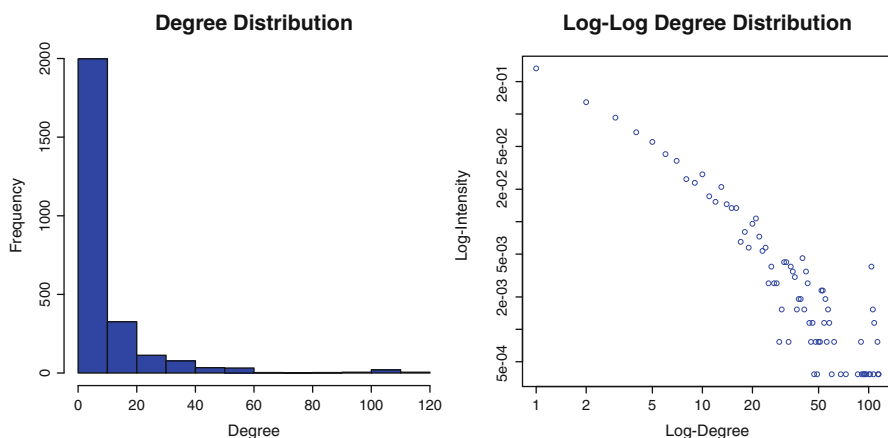


Fig. 4.2 Degree distribution for a network of protein–protein interactions in yeast. *Left*: Original scale; *Right*: log–log scale

From the plot on the right in Fig. 4.2 we see that there is a fairly linear decay in the log-frequency as a function of log-degree. While it is tempting to summarize the rate of this decay (i.e., the so-called degree exponent) using, for example, a simple linear regression, more sophisticated methods are preferable here. See [91, Chap. 4.2.1.1].

Beyond the degree distribution itself, it can be interesting to understand the manner in which vertices of different degrees are linked with each other. Useful in assessing this characteristic is the notion of the average degree of the *neighbors* of a given vertex. For example, a plot of average neighbor degree versus vertex degree in the yeast data, as shown in Fig. 4.3, suggests that while there is a tendency for vertices of higher degrees to link with similar vertices, vertices of lower degree tend to link with vertices of both lower and higher degrees.

```
#4.8 1 > a.nn.deg.yeast <- graph.knn(yeast, V(yeast))$knn
2 > plot(d.yeast, a.nn.deg.yeast, log="xy",
3 +     col="goldenrod", xlab=c("Log Vertex Degree"),
4 +     ylab=c("Log Average Neighbor Degree"))
```

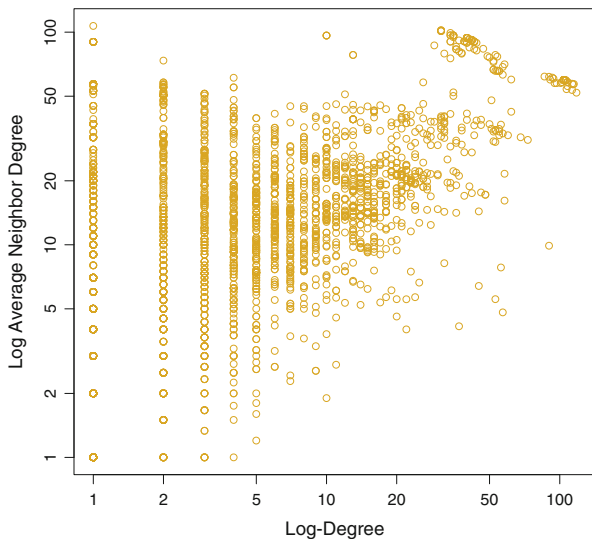


Fig. 4.3 Average neighbor degree versus vertex degree (log–log scale) for the yeast data

4.2.2 Vertex Centrality

Many questions that might be asked about a vertex in a network graph essentially seek to understand its ‘importance’ in the network. Which actors in a social network seem to hold the ‘reins of power’? How authoritative does a particular page in the World Wide Web seem to be considered? The deletion of which genes in a gene regulatory network is likely to be lethal to the corresponding organism? How critical is a given router in an Internet network to the flow of traffic? Measures of *centrality* are designed to quantify such notions of ‘importance’ and thereby facilitate the answering of such questions.

There are a vast number of different centrality measures that have been proposed over the years. We have already encountered what is arguably the most widely used measure of vertex centrality: vertex degree. Here we will focus our discussion primarily around the most common versions of three other classic types of vertex centrality measures—typically termed closeness, betweenness, and eigenvector centrality, respectively.

Closeness centrality measures attempt to capture the notion that a vertex is ‘central’ if it is ‘close’ to many other vertices. The standard approach, introduced by Sabidussi [127], is to let the centrality vary inversely with a measure of the total distance of a vertex from all others,

$$c_{Cl}(v) = \frac{1}{\sum_{u \in V} \text{dist}(v, u)}, \quad (4.1)$$

where $\text{dist}(v, u)$ is the geodesic distance between the vertices $u, v \in V$. Often, for comparison across graphs and with other centrality measures, this measure is normalized to lie in the interval $[0, 1]$, through multiplication by a factor $N_v - 1$.

Betweenness centrality measures are aimed at summarizing the extent to which a vertex is located ‘between’ other pairs of vertices. These centralities are based upon the perspective that ‘importance’ relates to where a vertex is located with respect to the paths in the network graph. If we picture those paths as the routes by which, say, communication of some sort or another takes place, vertices that sit on many paths are likely more critical to the communication process. The most commonly used betweenness centrality, introduced by Freeman [59], is defined as

$$c_B(v) = \sum_{s \neq t \neq v \in V} \frac{\sigma(s, t|v)}{\sigma(s, t)}, \quad (4.2)$$

where $\sigma(s, t|v)$ is the total number of shortest paths between s and t that pass through v , and $\sigma(s, t)$ is the total number of shortest paths between s and t (regardless of whether or not they pass through v). In the event that shortest paths are unique, $c_B(v)$ just counts the number of shortest paths going through v . This centrality measure can be restricted to the unit interval through division by a factor of $(N_v - 1)(N_v - 2)/2$.

Finally, other centrality measures are based on notions of ‘status’ or ‘prestige’ or ‘rank.’ That is, they seek to capture the idea that the more central the neighbors of a vertex are, the more central that vertex itself is. These measures are inherently implicit in their definition and typically can be expressed in terms of eigenvector solutions of appropriately defined linear systems of equations. There are many such *eigenvector centrality* measures. For example, Bonacich [20], following work of Katz [88] and others, defined a centrality measure of the form

$$c_{Ei}(v) = \alpha \sum_{\{u, v\} \in E} c_{Ei}(u). \quad (4.3)$$

The vector $\mathbf{c}_{Ei} = (c_{Ei}(1), \dots, c_{Ei}(N_v))^T$ is the solution to the eigenvalue problem $\mathbf{A}\mathbf{c}_{Ei} = \alpha^{-1}\mathbf{c}_{Ei}$, where $\mathbf{A}$ is the adjacency matrix for the network graph G . Bonacich [20] argues that an optimal choice of α^{-1} is the largest eigenvalue of $\mathbf{A}$, and hence $\mathbf{c}_{Ei}$ is the corresponding eigenvector. When G is undirected and connected, the largest eigenvalue of $\mathbf{A}$ will be simple and its eigenvector will have entries that are all nonzero and share the same sign. Convention is to report the absolute values of these entries, which will automatically lie between 0 and 1 by the orthonormality of eigenvectors.

An intuitively appealing way of displaying vertex centralities (for networks of small to moderate size) is to use a radial layout, with more central vertices located closer to the center. The function `gplot.target`, in the package **sna**, can be used for this purpose. For example,

```
#4.9 1 > A <- get.adjacency(karate, sparse=FALSE)
2 > library(network)
3 > g <- network::as.network.matrix(A)
4 > library(sna)
5 > sna::gplot.target(g, degree(g), main="Degree",
6 +   circ.lab = FALSE, circ.col="skyblue",
7 +   usearrows = FALSE,
8 +   vertex.col=c("blue", rep("red", 32), "yellow"),
9 +   edge.col="darkgray")
```

produces a visualization of degree centrality for the karate club network, as shown in Fig. 4.4. The visualizations of the other three centralities shown in this figure are produced similarly, replacing the argument `degree(g)` by the arguments `closeness(g)`, `betweenness(g)`, and `evcent(g)$vector`, respectively.¹

The plots in Fig. 4.4 illustrate how these four measures of centrality indeed capture different notions of ‘central’. Note that the relative centralities of the two arguably most important figures in this club—the head teacher and the administrator—vary with choice of measure, from nearly indistinct under degree centrality to most distinct under betweenness centrality. Furthermore, under the latter choice of centrality, the other actors in the network appear to be maximally distinct from those two as well. This last characteristic is a function of the largely star-like nature of this network, in which relatively few vertices v have shortest paths between pairs of other vertices s and t passing through v .

Extensions of these centrality measures from undirected to directed graphs are straightforward. However, in the latter case, there are in addition other useful options. For example, the HITS algorithm, based on the concept of ‘hubs and authorities,’ as introduced by Kleinberg [90] in the context of the World Wide Web, characterizes the importance of so-called hub vertices by how many authority vertices they point to, and so-called authority vertices by how many hubs point to them. Specifically, given an adjacency matrix $\mathbf{A}$ for a directed graph, hubs are determined according to the eigenvector centrality of the matrix $\mathbf{M}_{hub} = \mathbf{A}\mathbf{A}^T$, and authorities, according to that of $\mathbf{M}_{auth} = \mathbf{A}^T\mathbf{A}$.

Applying these measures to the AIDS blog network indicates, as seen in Fig. 4.5, that only six of the 146 blogs in this network play the role of a hub, while the vast majority of the vertices (including some of the hubs) play the role of an authority.

```
#4.10 1 > l <- layout.kamada.kawai(aidsblog)
2 > plot(aidsblog, layout=l, main="Hubs", vertex.label="",
3 +   vertex.size=10 * sqrt(hub.score(aidsblog)$vector))
4 > plot(aidsblog, layout=l, main="Authorities",
5 +   vertex.label="", vertex.size=10 *
6 +   sqrt(authority.score(aidsblog)$vector))
```

¹ Note that **igraph** and **sna** use different formats for storing network data objects. Here we handle the conversion by first producing an adjacency matrix for the karate club object `karate`.

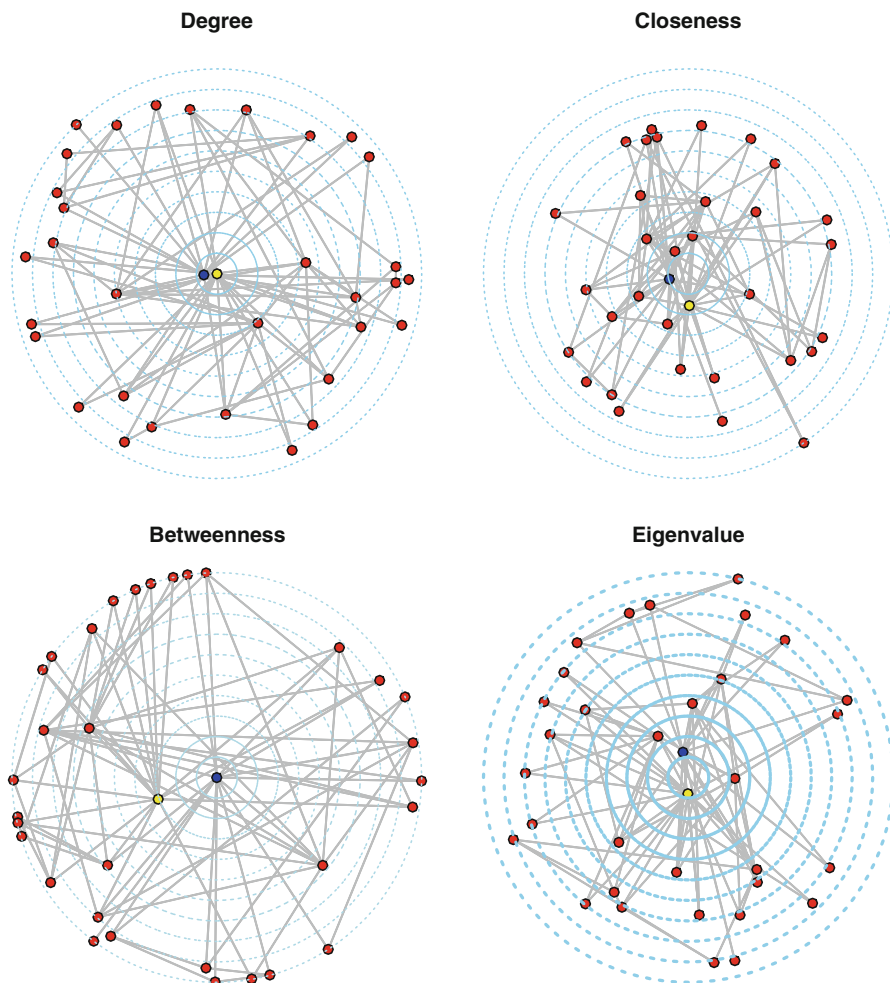


Fig. 4.4 Target plots showing various vertex centralities for the karate club network. The head teacher and instructor are indicated with *blue* and *yellow*, respectively

4.2.3 Characterizing Edges

All of the summary measures discussed so far (i.e., degree and other, more general, notions of centrality) are for vertices, as it seems to be most common in practice that questions of importance are in regard to the vertices of a graph. But some questions are more naturally associated with edges. For example, we might ask which ties in a social network are most important for the spread of, say, information or rumors. Edge betweenness centrality—which extends vertex betweenness centrality in a straightforward manner, by assigning to each edge a value that reflects the number of shortest paths traversing that edge—is a natural quantity to use here.

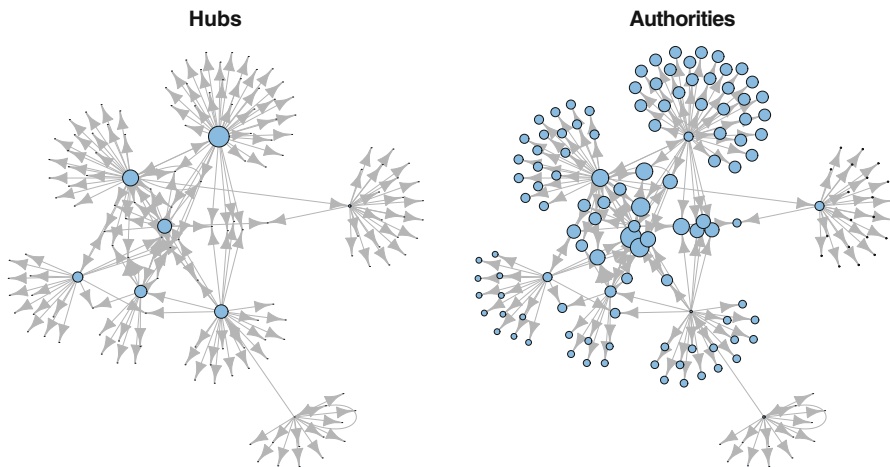


Fig. 4.5 AIDS blog network with vertex area proportional to hubs and authority centrality measures

Using edge betweenness with the karate network and examining, for instance, the edges with the three largest betweenness values

```
#4.11 1 > eb <- edge.betweenness(karate)
      2 > E(karate)[order(eb, decreasing=T)[1:3]]
      3 Edge sequence:
      4
      5 [53] John A -- Actor 20
      6 [14] Actor 20 -- Mr Hi
      7 [16] Actor 32 -- Mr Hi
```

we are led to note that actor 20 plays a key role from this perspective in facilitating the direct flow of information between the head instructor (Mr Hi, vertex 1) and the administrator (John A, vertex 34).

However, many other vertex centrality measures do not extend as easily. One way around this problem is to apply vertex centrality measures to the vertices in the line graph of a network graph G . The *line graph* of G , say $G' = (V', E')$, is obtained essentially by changing vertices of G to edges, and edges, to vertices, which in **igraph** may be accomplished using `line.graph`. More formally, the vertices $v' \in V'$ represent the original edges $e \in E$, and the edges $e' \in E'$ indicate that the two corresponding original edges in G were incident to a common vertex in G . See Chap. 3 of the edited volume of Brandes and Erlebach [22] for a brief discussion of this approach.

4.3 Characterizing Network Cohesion

A great many questions in network analysis boil down to questions involving *network cohesion*, the extent to which subsets of vertices are cohesive—or ‘stuck together’—with respect to the relation defining edges in the network graph. Do friends of a given actor in a social network tend to be friends of one another as well? What collections of proteins in a cell appear to work closely together? Does the structure of the pages in the World Wide Web tend to separate with respect to distinct types of content? What portion of a measured Internet topology would seem to constitute the ‘backbone’?

There are many ways that we can define network cohesion, depending on the context of the question being asked. Definitions differ, for example, in scale, ranging from local (e.g., triads) to global (e.g., giant components), and also in the extent to which they are specified explicitly (e.g., cliques) versus implicitly (e.g., ‘clusters’ or ‘communities’). In this section we discuss a number of common ways to define and summarize ‘cohesion’ in a network graph.

4.3.1 Subgraphs and Censuses

One approach to defining network cohesion is through specification of a certain subgraph(s) of interest. The canonical example of such a subgraph is that of a clique. Recall that cliques are complete subgraphs and hence are subsets of vertices that are fully cohesive, in the sense that all vertices within the subset are connected by edges.

A census of cliques of all sizes can provide some sense of a ‘snapshot’ of how structured a graph is.

```
#4.12 1 > table(sapply(cliques(karate), length))
      2
      3   1   2   3   4   5
      4 34  78 45 11   2
```

For the karate network a census of this sort reflects that there are 34 nodes (cliques of size one) and 78 edges (cliques of size two), followed by 45 triangles (cliques of size three). The prevalence of the latter are quite evident in the visualizations we have seen (e.g., Fig. 3.5). We also see that the largest cliques are of size five, of which there are only two. These two both involve four actors in common, including actor 1, i.e., the head instructor.

```
#4.13 1 > cliques(karate)[sapply(cliques(karate), length) == 5]
      2 [[1]]
      3 [1]  1  2  3  4  8
      4
      5 [[2]]
      6 [1]  1  2  3  4 14
```

Note that there is some redundancy in this analysis, in that the cliques of larger sizes necessarily include cliques of smaller sizes. A *maximal* clique is a clique that

is not a subset of a larger clique. In the karate network, the two largest cliques (formally called maximum cliques) are maximal, while, for example, the same can be said of only two of the 11 cliques of size four.

```
#4.14 1 > table(sapply(maximal.cliques(karate), length))
      2
      3  2  3  4  5
      4 11 21  2  2
```

In practice, large cliques are relatively rare, as they necessarily require that a graph G itself be fairly dense, while real-world networks are often sparse. For example, in the network of protein–protein interactions in yeast encountered earlier in this chapter, despite being roughly two orders of magnitude larger than the karate network, the size of the largest clique (formally, the *clique number*) is nevertheless comparatively small.

```
#4.15 1 > clique.number(yeast)
      2 [1] 23
```

Various weakened notions of cliques exist. For example, a k -core of a graph G is a subgraph of G for which all vertex degrees are at least k , and such that no other subgraph obeying the same condition contains it (i.e., it is maximal in this property). The notion of cores is particularly popular in visualization, as it provides a way of decomposing a network into ‘layers’, in the sense of an onion. Such decompositions can be combined in a particularly effective manner with a radial layout (e.g., using a target plot). Figure 4.6 shows the karate network represented in this way.

```
#4.16 1 > cores <- graph.coreness(karate)
      2 > sna::gplot.target(g, cores, circ.lab = FALSE,
      3 +                   circ.col="skyblue", usearrows = FALSE,
      4 +                   vertex.col=cores, edge.col="darkgray")
      5 > detach("package:network")
      6 > detach("package:sna")
```

See [91, Chap. 3.5.2] for an illustration of the same principles in the context of a much larger network, representing a nontrivial portion of the router-level logical topology in the Internet.

Beyond cliques and their variants, there are other classes of subgraphs of common interest in defining network cohesion. Two fundamental quantities, going back to early work in social network analysis [74, 43], are dyads and triads. Dyads are pairs of vertices and, in directed graphs, may take on three possible states: null (no directed edges), asymmetric (one directed edge), or mutual (two directed edges). Similarly, triads are triples of vertices and may take on 16 possible states, ranging from the null subgraph to the subgraph in which all three dyads formed by the vertices in the triad have mutual directed edges.

A census of the possible states of these two classes of subgraphs, i.e., counting how many times each state is observed in a graph G , can yield insight into the nature of the connectivity in the graph. For example, in the AIDS blog network, we see that the vast majority of the dyads are null and, of those that are non-null, almost all are

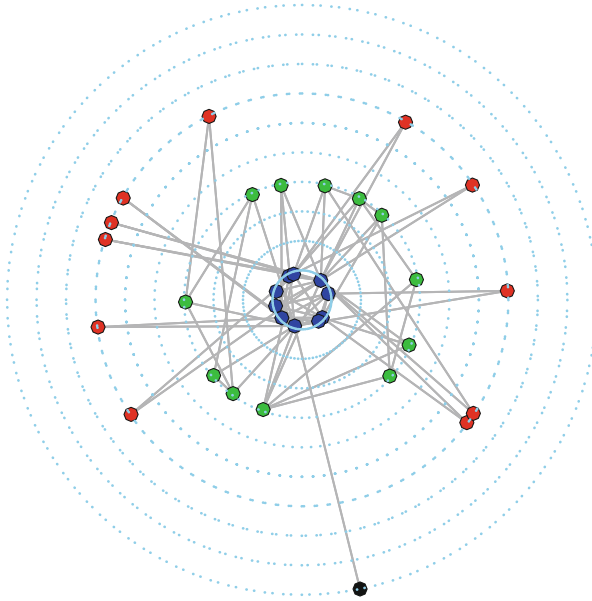


Fig. 4.6 Visual representation of the k -core decomposition of the karate network. Vertices of coreness one (*black*), two (*red*), three (*green*), and four (*blue*) are shown at successively smaller distances from the center, with the same distance for vertices within each core

asymmetric, indicating a decided one-sidedness to the manner in which blogs in this network reference each other.

```
#4.17 1 > aidsblog <- simplify(aidsblog)
      2 > dyad.census(aidsblog)
      3 $mut
      4 [1] 3
      5
      6 $asym
      7 [1] 177
      8
      9 $null
     10 [1] 10405
```

This analysis² is consistent with the observations from our earlier analysis of hubs and authorities in this network, as shown in Fig. 4.5.

More generally, we note that a census may in principle be conducted of *any* subgraph(s) of interest. Small connected subgraphs of interest are commonly termed *motifs*. The notion of motifs is particularly popular in the study of biological networks, where arguments often are made linking such network substructures to biological function. Examples include subgraphs with a fan-like structure (i.e., multiple

² Note that we first remove self-loops (of which the original AIDS blog network has three), since the notion of mutuality is well-defined only for dyads.

directed edges emanating from a single vertex) and feedforward loops (i.e., three directed edges, among three vertices, of the form $\{u, v\}$, $\{v, w\}$, and $\{u, w\}$). Note that in large network graphs the enumeration of all occurrences of a given motif can be quite time consuming. For this reason, sampling methods are sometimes used, with motif counts estimated from the sample. The **igraph** function `graph.motifs` has this option.

4.3.2 Density and Related Notions of Relative Frequency

The characterizations of network cohesion described so far proceed by first stating a pre-specified notion of substructure and then looking to see whether it occurs in a graph G and, if so, where and how often. More generally, the related concept of relative frequency can be applied in various useful ways.

The *density* of a graph is the frequency of realized edges relative to potential edges. For example, in a (undirected) graph G with no self-loops and no multiple edges, the density of a subgraph $H = (V_H, E_H)$ is

$$\text{den}(H) = \frac{|E_H|}{|V_H|(|V_H| - 1)/2}. \quad (4.4)$$

The value of $\text{den}(H)$ will lie between zero and one and provides a measure of how close H is to being a clique. In the case that G is a directed graph, the denominator in (4.4) is replaced by $|V_H|(|V_H| - 1)$.

The arguably simple concept of density is made interesting through the freedom we have in the choice of subgraph H defining (4.4). For instance, taking $H = G$ yields the density of the overall graph G . Conversely, taking $H = H_v$ to be the set of neighbors of a vertex $v \in V$, and the edges between them, yields a measure of density in the immediate neighborhood of v .

Applying these ideas to the karate network, for example, we see that the subgraphs corresponding to each of the instructor and the administrator, in union with their immediate respective neighborhoods—i.e., the ego-centric networks around vertices 1 and 34—are noticeably more dense than the overall network.

```
#4.18 1 > ego.instr <- induced.subgraph(karate,
2 +   neighborhood(karate, 1, 1)[[1]])
3 > ego.admin <- induced.subgraph(karate,
4 +   neighborhood(karate, 1, 34)[[1]])
5 > graph.density(karate)
6 [1] 0.1390374
7 > graph.density(ego.instr)
8 [1] 0.25
9 > graph.density(ego.admin)
10 [1] 0.2091503
```

This observation is consistent with the disparity in the number of within-versus between-faction edges in this network, evident in the visualization of Fig. 3.5.

Relative frequency also is used in defining notions of ‘clustering’ in a graph. For example, the standard use of the term *clustering coefficient* typically refers to the quantity

$$cl_T(G) = \frac{3\tau_\Delta(G)}{\tau_3(G)}, \quad (4.5)$$

where $\tau_\Delta(G)$ is the number of triangles in the graph G , and $\tau_3(G)$, the number of connected triples (i.e., a subgraph of three vertices connected by two edges, also sometimes called a 2-star). The value $cl_T(G)$ is alternatively called the *transitivity* of the graph, and is a standard quantity of interest in the social network literature, where it is also referred to as the ‘fraction of transitive triples.’

Note that $cl_T(G)$ is a measure of global clustering, summarizing the relative frequency with which connected triples close to form triangles. For example, in the karate network we see that only about one quarter of the connected triples close in this manner.

```
#4.19 1 > transitivity(karate)
      2 [1] 0.2556818
```

The local analogue of this measure can also be of interest. Let $\tau_\Delta(v)$ denote the number of triangles in G into which $v \in V$ falls, and $\tau_3(v) = \binom{d_v}{2}$, the number of connected triples in G for which the two edges are both incident to v . The local clustering coefficient is defined as $cl(v) = \tau_\Delta(v)/\tau_3(v)$, for those vertices v with $\tau_3(v) > 0$. In the case of the instructor and administrator of the karate network, for example, we see that their local clustering is only 50–60% that of the clustering for the network as a whole.

```
#4.20 1 > transitivity(karate, "local", vids=c(1,34))
      2 [1] 0.1500000 0.1102941
```

A concept unique to directed graphs is that of reciprocity, i.e., the extent to which there is reciprocation among ties in a directed network. There are two main approaches to capturing this notion, distinguished by whether the unit of interest in computing relative frequencies is that of dyads or directed edges. In the case that dyads are used as units, reciprocity is defined to be the number of dyads with reciprocated (i.e., mutual) directed edges divided by the number of dyads with a single, unreciprocated edge. Alternatively, reciprocity is defined as the total number of reciprocated edges divided by the total number of edges.

In the AIDS blog network, the reciprocity is quite low by either definition.

```
#4.21 1 > reciprocity(aidsblog, mode="default")
      2 [1] 0.03278689
      3 > reciprocity(aidsblog, mode="ratio")
      4 [1] 0.01666667
```

Recalling our dyad census of the same network, this result is not surprising.

4.3.3 Connectivity, Cuts, and Flows

A basic question of interest is whether a given graph separates into distinct sub-graphs. If it does not, we might seek to quantify how close to being able to do so it is. Intimately related to such issues are questions associated with the flow of ‘information’ in the graph. In this section, we discuss various ways to quantify such notions.

Recall that a graph G is said to be connected if every vertex is reachable from every other (i.e., if for any two vertices, there exists a walk between the two), and that a connected component of a graph is a maximally connected subgraph. Often it is the case that one of the connected components in a graph G dominates the others in magnitude, in that it contains the vast majority of the vertices in G . Such a component is called, borrowing terminology from random graph theory, the *giant component*.

Recall, for example, our network of protein interactions in yeast. The network graph of 2617 vertices is not connected.

```
#4.22 1 > is.connected(yeast)
      2 [1] FALSE
```

A census of all connected components within this graph, however, shows that there clearly is a giant component.

```
#4.23 1 > comps <- decompose.graph(yeast)
      2 > table(sapply(comps, vcount))
      3
      4      2      3      4      5      6      7 2375
      5     63     13      5      6      1      3      1
```

This single component contains $2375/2617 \approx 90\%$ of the vertices in the network. In contrast, none of the other components alone contain even 1%. In practice, often attention would be restricted to this giant component alone in carrying out further analysis and modeling.

```
#4.24 1 > yeast.gc <- decompose.graph(yeast)[[1]]
```

For example, a celebrated characteristic observed in the giant component of many real-world networks is the so-called *small world* property, which refers to the situation wherein (a) the shortest-path distance between pairs of vertices is generally quite small, but (b) the clustering is relatively high. In our network of protein–protein interactions in yeast, we see that the average path length in the giant component is barely greater than five

```
#4.25 1 > average.path.length(yeast.gc)
      2 [1] 5.09597
```

and even the longest of paths is not much bigger.

```
#4.26 1 > diameter(yeast.gc)
      2 [1] 15
```

Hence, the shortest-path distance in this network scales more like $\log N_v$ rather than N_v , and therefore is considered ‘small’. At the same time, the clustering in this network is relatively large

```
#4.27 1 > transitivity(yeast.gc)
      2 [1] 0.4686663
```

indicating that close to 50 % of connected triples close to form triangles.

A somewhat more refined notion of connectivity than components derives from asking whether, if an arbitrary subset of k vertices (edges) is removed from a graph, the remaining subgraph is connected. The concepts of vertex- and edge-connectivity, and the related concepts of vertex- and edge-cuts, help to make such notions precise.

A graph G is called k -vertex-connected if (i) the number of vertices $N_v > k$, and (ii) the removal of any subset of vertices $X \subset V$ of cardinality $|X| < k$ leaves a subgraph that is connected. Similarly, G is called k -edge-connected if (i) $N_v \geq 2$, and (ii) the removal of any subset of edges $Y \subseteq E$ of cardinality $|Y| < k$ leaves a subgraph that is connected. The *vertex (edge) connectivity* of G is the largest integer such that G is k -vertex- (k -edge-) connected. It can be shown that the vertex connectivity is bounded above by the edge connectivity, which in turn is bounded above by the minimum degree $d_{\min}$ among vertices in G .

In the case of the giant component of the yeast network, the vertex and edge connectivity are both equal to one.

```
#4.28 1 > vertex.connectivity(yeast.gc)
      2 [1] 1
      3 > edge.connectivity(yeast.gc)
      4 [1] 1
```

Thus it requires the removal of only a single well-chosen vertex or edge in order to break this subgraph into additional components.

If the removal of a particular set of vertices (edges) in a graph disconnects the graph, that set is called a *vertex-cut (edge-cut)*. A single vertex that disconnects the graph is called a *cut vertex*, or sometimes an *articulation point*. Identification of such vertices can provide a sense of where a network is vulnerable (e.g., in the sense of an attack, where disconnecting produces undesired consequences, such as a power outage in an energy network). In the giant component of the yeast network, almost 15 % of the vertices are cut vertices.

```
#4.29 1 > yeast.cut.vertices <- articulation.points(yeast.gc)
      2 > length(yeast.cut.vertices)
      3 [1] 350
```

A fundamental result in graph theory, known as Menger’s theorem, essentially states that a nontrivial graph G is k -vertex (k -edge) connected if and only if all pairs of distinct vertices $u, v \in V$ can be connected by k vertex-disjoint (edge-disjoint) paths. This result relates the robustness of a graph in the face of removal of its vertices (edges) to the richness of distinct paths running throughout it. A graph with low vertex (edge) connectivity therefore can have the paths, and hence any ‘information’ flowing over those paths, disrupted by removing an appropriate choice of a correspondingly small number of vertices (edges). Functions like `shortest.paths`,

`graph.maxflow`, and `graph.mincut` in **igraph** can be used to calculate quantities relevant to this connection between cuts and flows.

Most of the concepts introduced above extend to the case of directed graphs in a straightforward manner. For example, recall that a digraph G is weakly connected if its underlying graph is connected, and it is called strongly connected if every vertex v is reachable from every other vertex u by a directed walk. A strongly connected component is a subgraph that is strongly connected. The concept of vertex-connectivity (edge-connectivity) extends analogously by requiring that the graphs remaining after removal of vertices (edges) be strongly connected. Similarly, the notion of cuts and flows remain essentially unchanged, except that now there is a directionality designated.

Note that the distinction between strong and weak connectivity can be severe for some digraphs. For example, the AIDS blog network is weakly connected

```
#4.30 1 > is.connected(aidsblog, mode=c("weak"))
      2 [1] TRUE
```

but not strongly connected.

```
#4.31 1 > is.connected(aidsblog, mode=c("strong"))
      2 [1] FALSE
```

And while there does exist a strongly connected component within the graph, there is only one and it has only four vertices.

```
#4.32 1 > aidsblog.scc <- clusters(aidsblog, mode=c("strong"))
      2 > table(aidsblog.scc$ccsize)
      3
      4      1      4
      5 142      1
```

4.4 Graph Partitioning

Partitioning—broadly speaking—refers to the segmentation of a set of elements into ‘natural’ subsets. More formally, a *partition* $\mathcal{C} = \{C_1, \dots, C_K\}$ of a finite set S is a decomposition of S into K disjoint, nonempty subsets C_k such that $\cup_{k=1}^K C_k = S$. In the analysis of network graphs, partitioning is a useful tool for finding, in an unsupervised fashion, subsets of vertices that demonstrate a ‘cohesiveness’ with respect to the underlying relational patterns.

A ‘cohesive’ subset of vertices generally is taken to refer to a subset of vertices that (i) are well connected among themselves, and at the same time (ii) are relatively well separated from the remaining vertices. More formally, graph partitioning algorithms typically seek a partition $\mathcal{C} = \{C_1, \dots, C_K\}$ of the vertex set V of a graph $G = (V, E)$ in such a manner that the sets $E(C_k, C_{k'})$ of edges connecting vertices in C_k to vertices in $C_{k'}$ are relatively small in size compared to the sets $E(C_k) = E(C_k, C_k)$ of edges connecting vertices within the C_k .

This problem of *graph partitioning* is also commonly referred to as *community detection* in the complex networks literature. The development of methods

for community detection has been—and continues to be—a highly active area of research. Here we illustrate through the use of two well-established classes of methods—those based on adaptations of hierarchical clustering and those based on spectral partitioning. For an extensive survey of this area, see [95].

4.4.1 Hierarchical Clustering

A great many methods for graph partitioning are essentially variations on the more general concept of *hierarchical clustering* used in data analysis.³ There are numerous techniques that have been proposed for the general clustering problem, differing primarily in (i) how they evaluate the quality of proposed clusterings and (ii) the algorithms by which they seek to optimize that quality. See Jain, Murty, and Flynn [82], for example. These methods take a greedy approach to searching the space of all possible partitions $\mathcal{C}$, by iteratively modifying successive candidate partitions.

Hierarchical methods are classified as either *agglomerative*, being based on the successive coarsening of partitions through the process of merging, or *divisive*, being based on the successive refinement of partitions through the process of splitting. At each stage, the current candidate partition is modified in a way that minimizes a specified measure of cost. In agglomerative methods, the least costly merge of two previously existing partition elements is executed, whereas in divisive methods, it is the least costly split of a single existing partition element into two that is executed.

The measure of cost incorporated into a hierarchical clustering method used in graph partitioning should reflect our sense of what defines a ‘cohesive’ subset of vertices. There are many cost measures that have been proposed. A particularly popular measure is that of *modularity* [119]. Let $\mathcal{C} = \{C_1, \dots, C_K\}$ be a given candidate partition and define $f_{ij} = f_{ij}(\mathcal{C})$ to be the fraction of edges in the original network that connect vertices in C_i with vertices in C_j . The modularity of $\mathcal{C}$ is the value

$$\text{mod}(\mathcal{C}) = \sum_{k=1}^K [f_{kk}(\mathcal{C}) - f_{kk}^*]^2, \quad (4.6)$$

where f_{kk}^* is the expected value of f_{kk} under some model of random edge assignment. Most commonly, f_{kk}^* is defined to be $f_{k+}f_{+k}$, where f_{k+} and f_{+k} are the k th row and column sums of $\mathbf{f}$, the $K \times K$ matrix⁴ formed by the entries f_{ij} . This choice corresponds to a model in which a graph is constructed to have the same degree distribution as G , but with edges otherwise placed at random, without respect to

³ Note that ‘clustering’ as used here, which is standard terminology in the broader data analysis community, differs from ‘clustering’ as used in Sect. 4.3.2, which arose in the social network community, in reference to the coefficient cl_T used to summarize the relative density of triangles among connected triples.

⁴ Note that for undirected graphs this matrix will be symmetric, and hence $f_{k+} = f_{+k}$; for directed graphs, however, $\mathbf{f}$ can be asymmetric.

the underlying partition elements dictated by $\mathcal{C}$. Large values of the modularity are therefore taken to suggest that $\mathcal{C}$ captures nontrivial ‘group’ structure, beyond that expected to occur under the random assignment of edges.

In principle the optimization of the modularity in (4.6) requires a search over all possible partitions $\mathcal{C}$, which is prohibitively expensive in networks of moderate size and larger. A fast, greedy approach to optimization has been proposed in [30], in the form of an agglomerative hierarchical clustering algorithm, and implemented in **igraph** as `fastgreedy.community`. The result of this and related community detection methods in **igraph** is to produce an object of the class *communities*, which can then serve as input to various other functions.

Applying this method to the karate network,

```
#4.33 1 > kc <- fastgreedy.community(karate)
```

we find that the method has declared there to be three communities.

```
#4.34 1 > length(kc)
2 [1] 3
3 > sizes(kc)
4 Community sizes
5 1 2 3
6 18 11 5
```

Based on what we know of this network, and examining the community membership

```
#4.35 1 > membership(kc)
2 Mr Hi Actor 2 Actor 3 Actor 4 Actor 5 Actor 6
3 2 2 2 2 3 3
4 Actor 7 Actor 8 Actor 9 Actor 10 Actor 11 Actor 12
5 3 2 1 1 3 2
6 Actor 13 Actor 14 Actor 15 Actor 16 Actor 17 Actor 18
7 2 2 1 1 3 2
8 Actor 19 Actor 20 Actor 21 Actor 22 Actor 23 Actor 24
9 1 2 1 2 1 1
10 Actor 25 Actor 26 Actor 27 Actor 28 Actor 29 Actor 30
11 1 1 1 1 1 1
12 Actor 31 Actor 32 Actor 33 John A
13 1 1 1 1
```

it would be reasonable to conjecture that the largest community of 18 members is centered around the administrator (i.e., John A, vertex ID 34), while the second largest community of 11 members is centered around the head instructor (i.e., Mr Hi, vertex ID 1). The visual representation obtained by plotting the network with these community designations indicated,

```
#4.36 1 > plot(kc, karate)
```

as seen on the left in Fig. 4.7, provides further support for this conjecture.

Whether agglomerative or divisive, when used for network graph partitioning, hierarchical clustering methods actually produce, as the name indicates, an entire hierarchy of nested partitions of the graph, not just a single partition. These partitions can range fully between the two trivial partitions $\{\{v_1\}, \dots, \{v_{N_v}\}\}$ and V .

Agglomerative methods begin with the former of these two, while divisive methods begin with the latter. The resulting hierarchy typically is represented in the form of a tree, called a *dendrogram*.

An example of a dendrogram is shown on the right in Fig. 4.7, for our hierarchical partitioning of the karate network, using the **igraph** function `dendPlot`.

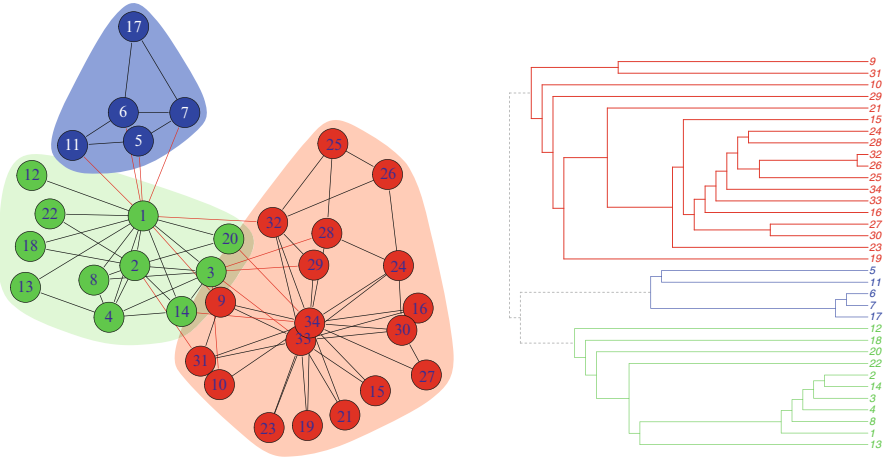


Fig. 4.7 *Left*: Partitioning of the karate network obtained from fast agglomerative hierarchical clustering approximation to optimization of modularity. *Right*: The corresponding dendrogram for this partitioning

```
#4.37 1 > library(ape)
      2 > dendPlot(kc, mode="phylo")
```

The package **ape** is called to facilitate the mode ‘phylo’ in `dendPlot`, which uses tools from the former, designed for display of phylogenetic trees, in rendering the dendrogram.

4.4.2 Spectral Partitioning

Another common approach to graph partitioning is to exploit results in spectral graph theory that associate the connectivity of a graph G with the eigen-analysis of certain matrices. Here we look at one popular approach, based on analysis of the so-called Laplacian of a graph. For general background on spectral graph theory, see the monograph by Chung [28].

The *graph Laplacian* of a graph G , with adjacency matrix $\mathbf{A}$, is a matrix $\mathbf{L} = \mathbf{D} - \mathbf{A}$, where $\mathbf{D} = \text{diag}[(d_v)]$ is a diagonal matrix with elements $D_{vv} = d_v$ the entries of the degree sequence of G . A formal result in spectral graph theory (e.g., [64, Lemma 13.1.1]) states that a graph G will consist of K connected components if and only if $\lambda_1(\mathbf{L}) = \dots = \lambda_K(\mathbf{L}) = 0$ and $\lambda_{K+1}(\mathbf{L}) > 0$, where $\lambda_1 \leq \dots \leq \lambda_{N_v}$ are

the (not necessarily distinct) eigenvalues of $\mathbf{L}$, ordered from small to large. Hence, the number of components in a graph is directly related to the number of non-zero eigenvalues of the graph Laplacian.

The smallest eigenvalue of $\mathbf{L}$ can be shown to be identically equal to zero, with corresponding eigenvector $\mathbf{x}_1 = (1, \dots, 1)^T$. Therefore, for example, if we suspect a graph G to consist of ‘nearly’ $K = 2$ components, and hence to be a good candidate for bisection, we might expect $\lambda_2(\mathbf{L})$ to be close to zero. In fact, such expectations are reasonable, as the value λ_2 is closely related to a number of measures of graph connectivity and structure. In particular, these relationships indicate that the smaller λ_2 the more amenable the graph is to being separated into two subgraphs by severing a relatively small number of edges between the two. See the survey by Mohar [113] or, more recently, the volume edited by Brandes and Erlebach [22, Chap. 14].

Fiedler [56], the first to associate λ_2 with the connectivity of a graph, suggested partitioning vertices by separating them according to the sign of their entries in the corresponding eigenvector $\mathbf{x}_2$. The result is to produce two subsets of vertices (a so-called *cut*)

$$S = \{v \in V : \mathbf{x}_2(v) \geq 0\} \quad \text{and} \quad \bar{S} = \{v \in V : \mathbf{x}_2(v) < 0\}. \quad (4.7)$$

The vector $\mathbf{x}_2$ is hence often called the *Fiedler vector*, and the corresponding eigenvalue λ_2 , the *Fiedler value*.

We illustrate using the karate network again. It is straightforward to do the necessary eigen-analysis.

```
#4.38 1 > k.lap <- graph.laplacian(karate)
      2 > eig.anal <- eigen(k.lap)
```

We plot the eigenvalues of the graph Laplacian in Fig. 4.8

```
#4.39 1 > plot(eig.anal$values, col="blue",
      2 +   ylab="Eigenvalues of Graph Laplacian")
```

and see that (a) there is only one eigenvalue exactly equal to zero (as expected, since this network is connected) and (b) the second smallest eigenvalue λ_2 is quite close to zero. Extracting the Fiedler vector

```
#4.40 1 > f.vec <- eig.anal$vectors[, 33]
```

and plotting the entries of that vector versus actor number,

```
#4.41 1 > faction <- get.vertex.attribute(karate, "Faction")
      2 > f.colors <- as.character(length(faction))
      3 > f.colors[faction == 1] <- "red"
      4 > f.colors[faction == 2] <- "cyan"
      5 > plot(f.vec, pch=16, xlab="Actor Number",
      6 +   ylab="Fiedler Vector Entry", col=f.colors)
      7 > abline(0, 0, lwd=2, col="lightgray")
```

we find that this spectral method exactly captures the partitioning of the network indicated by the faction labels.

In general, of course, we may well expect that a network should be partitioned into more than just two subgraphs. The method of spectral partitioning just described can be applied iteratively, first partitioning a graph into two subgraphs, then each of those two subgraphs into further subgraphs, and so on. Ideally, however, it is desirable that such iterations be aimed at optimizing some common objective function. Newman [117] proposes a method whose technical development parallels that

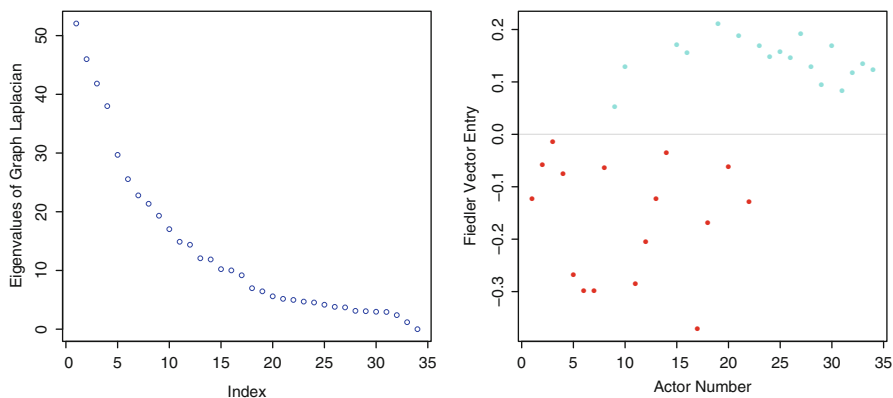


Fig. 4.8 *Left:* Eigenvalues of the graph Laplacian L for the karate network. *Right:* The accompanying Fiedler vector and its corresponding partition (indicated by the gray horizontal line). The two factions in the club (led by the instructor and the administrator, respectively) are indicated in red and cyan

of the spectral bisection method quite closely, but with a matrix related to modularity playing the role of the Laplacian L . This method is implemented in the function `leading.eigenvector.community` in **igraph**.

4.4.3 Validation of Graph Partitioning

Just as in the general problem of clustering data, the question of validation is important for graph partitioning—but often nontrivial. It is generally expected, where cohesive subsets of vertices are present in a network graph, that underlying these subsets there is some commonality in certain relevant characteristics (or attributes) of the vertices. For example, proteins found to cluster in a graph of protein interactions often participate in similar biological processes; similarly, actors found to cluster in a social network may share certain interests.

Graph partitioning may be viewed as a tool for discovering such subsets in the absence of knowledge of these characteristics. When we do have knowledge of some externally defined notion of class membership, it can be interesting to compare and contrast the resulting assignments with those deriving from graph partitioning. As an illustration, consider our network of protein–protein interactions in yeast.

These particular yeast data include the assignment of proteins to one of 13 functional classes (including “unknown”, denote by ‘U’).

```
#4.42 1 > func.class <- get.vertex.attribute(yeast.gc, "Class")
2 > table(func.class)
3 func.class
4      A      B      C      D      E      F      G      M      O      P      R      T      U
5  51   98  122  238   95  171   96  278  171  248   45  240  483
```

29	26	0	0	1	0	0	0	0	4	0	0	1	0	1
30	27	3	0	4	0	0	1	0	0	0	0	0	0	0
31	28	0	0	0	0	0	0	0	0	0	6	0	0	0
32	29	0	0	0	1	0	0	0	1	0	0	3	0	0
33	30	0	0	0	0	0	0	0	0	0	2	0	0	2
34	31	0	0	0	0	0	0	0	3	0	0	0	0	0

Some of the membership assignments resulting from our algorithm overlap quite strongly with individual functional classes. For example, 110 of the 182 proteins in the first community have the functional class “P” (indicating a role in protein synthesis), which suggests that the first community is largely capturing that class. On the other hand, the 733 proteins in the fourth community are spread through all functional classes (including 121 with unknown function), making this community decidedly less interpretable against this validation set.

4.5 Assortativity and Mixing

Selective linking among vertices, according to a certain characteristic(s), is termed *assortative mixing* in the social network literature. Measures that quantify the extent of assortative mixing in a given network have been referred to as *assortativity coefficients*, and are essentially variations on the concept of correlation coefficients.

Note that the vertex characteristics involved can be categorical, ordinal, or continuous. Consider the categorical case, and suppose that each vertex in a graph G can be labeled according to one of M categories. The assortativity coefficient in this setting is defined to be

$$r_a = \frac{\sum_i f_{ii} - \sum_i f_{i+} f_{+i}}{1 - \sum_i f_{i+} f_{+i}}, \quad (4.8)$$

where f_{ij} is the fraction of edges in G that join a vertex in the i th category with a vertex in the j th category, and f_{i+} and f_{+i} denote the i th marginal row and column sums, respectively, of the resulting matrix⁵ $\mathbf{f}$.

The value r_a lies between -1 and 1 . It is equal to zero when the mixing in the graph is no different from that obtained through a random assignment of edges that preserves the marginal degree distribution (see the related discussion surrounding (4.6) above). Similarly, it is equal to one when there is perfect assortative mixing (i.e., when edges only connect vertices of the same category). However, in the event that the mixing is perfectly disassortative, in the sense that every edge in the graph connects vertices of two different categories, the coefficient in (4.8) need not take the value -1 . See [116] for discussion.

Consider again our network of protein–protein interactions in yeast. The fact that physical binding of proteins is known to be directly relevant to functional classes

⁵ Note that these quantities are defined in complete analogy to the same quantities that underlie our definition of modularity.

suggests that there will frequently be strong assortative mixing in protein–protein interaction networks with respect to these classes as attributes. For instance, for the class ‘P’, representing proteins that are known to play a role in protein synthesis, we see an assortativity coefficient of nearly 0.5.

```
#4.45 1 > assortativity.nominal(yeast, (V(yeast)$Class=="P")+1,
      2 +   directed=FALSE)
      3 [1] 0.4965229
```

When the vertex characteristic of interest is continuous, rather than discrete, denote by (x_e, y_e) the values of that characteristic for the vertices joined by an edge $e \in E$. Then a natural candidate for quantifying the assortativity in this characteristic is just the Pearson correlation coefficient of the pairs (x_e, y_e) ,

$$r = \frac{\sum_{x,y} xy (f_{xy} - f_{x+} f_{+y})}{\sigma_x \sigma_y}. \quad (4.9)$$

Here the summation is over all unique observed pairs (x, y) , with f_{xy} , f_{x+} , and f_{+y} defined in analogy to the categorical case, and σ_x and σ_y are the standard deviations corresponding to the distribution of frequencies $\{f_{x+}\}$ and $\{f_{+y}\}$, respectively.

One common use of the assortativity coefficient r in (4.9) is in summarizing degree–degree correlation of adjacent vertices, when studying network structure. For the yeast network, we find that the degree correlation is positive and relatively large.

```
#4.46 1 > assortativity.degree(yeast)
      2 [1] 0.4610798
```

This observation is consistent with the analysis in Fig. 4.3.

4.6 Additional Reading

The array of tools and techniques available for the description and summary of structural characteristics of a network graph is quite large and still growing. Here in this chapter we have contented ourselves simply with presenting material on topics arguably at the core of this area. The volume edited by Brandes and Erlebach [22] provides a much more detailed—yet highly readable—presentation of most of the topics in this chapter, as well a number of others not covered here, from the perspective of computer science and applied graph theory. The book by Newman [118] presents a similarly detailed treatment from the perspective of statistical physics. Finally, in the literature on social networks, two canonical references are the small volume by Scott [129] and the much larger volume by Wasserman and Faust [144].